

Detailed Theory

1. Using Optional and Named Arguments

- Function arguments in Python can be **positional** or **named** (keyword arguments).
- **Optional arguments** have default values and do not need to be passed by the caller.
- Example of optional argument with default:

python

CopyEdit

```
def greet(name, msg="Hello"): print(f'{msg}, {name}!') greet("Komal")    # Uses default msg = "Hello" greet("Komal", "Hi")  # Overrides msg
```

- **Named arguments** allow specifying arguments by name, improving readability:

python

CopyEdit

```
greet(name="Komal", msg="Hi") greet(msg="Hey", name="Komal")
```

- Positional arguments must come before named arguments in function calls.

2. Using `type()`, `str()`, `dir()`, and Other Built-in Functions

- **`type(obj)`** returns the type/class of an object:

python

CopyEdit

```
type(5)    # <class 'int'> type("abc")  # <class 'str'>
```

-

str(obj) converts an object to its string representation:

python

CopyEdit

```
str(123)    # '123'
```

-

dir(obj) returns a list of valid attributes and methods for an object:

python

CopyEdit

`dir([])` # Lists methods of list objects

-

Other useful built-in functions include:

-

`len()` – length of collections

-

`id()` – unique identifier of an object

-

`help()` – shows documentation

-

`isinstance(obj, class)` – checks type compatibility

-

`hasattr(obj, attr)` – checks if attribute exists

3. Object References

- In Python, variables store **references** to objects, not the actual data itself.

- Assignment copies references, not objects:

python

CopyEdit

```
a = [1, 2] b = a b.append(3) print(a) # [1, 2, 3]
```

- This means changes via one reference affect the object seen by all references.

- For immutable types (e.g., int, str), reassignment creates new objects.
 - The id() function can be used to check if two variables reference the same object.
-

4. Socket with Python

- **Sockets** enable network communication between devices using IP addresses and ports.
- Python provides the socket module to create sockets for client-server communication.

Basic socket example:

- **Creating a TCP server:**

python

CopyEdit

```
import socket server_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) server_socket.bind(("localhost", 12345))
server_socket.listen(1) print("Waiting for connection...") client_socket, addr = server_socket.accept() print("Connected by", addr) data =
client_socket.recv(1024) print("Received:", data.decode()) client_socket.sendall(b"Hello Client") client_socket.close() server_socket.close()
```

-

Creating a TCP client:

python

CopyEdit

```
import socket client_socket = socket.socket(socket.AF_INET, socket.SOCK_STREAM) client_socket.connect(("localhost", 12345))
client_socket.sendall(b"Hello Server") data = client_socket.recv(1024) print("Received:", data.decode()) client_socket.close()
```

-

Key socket concepts:

- AF_INET: IPv4 addressing
- SOCK_STREAM: TCP protocol
- bind(): binds socket to address and port
- listen(): listens for incoming connections (server)
- accept(): accepts a connection (server)
- connect(): connects to a server (client)
- send() and recv(): send and receive data